Design, Simulation, and Hardware Implementation of a Pantograph Parallel Robot Drawing System

Yusr Mohammed, Noor Khaled, Reham Hassan, Khaled Masoud, Mohamed Ehab,
Fadi Hani, Zeineldine Mohamed, Mark Ashraf, Amr Ali, and Youssef Hussein
Faculty of Engineering, German International University, Cairo, Egypt
Course: Parallel Kinematics and Kinetics
Instructor: Dr. Khaled Tolba

***Abstract*—This paper presents the design, kinematic analysis, simulation, and hardware implementation of a planar pantograph parallel robot used as a drawing mechanism. The robot is modeled as a 5R planar parallel mechanism with two actuated revolute joints and two passive distal links connected to a common end-effector point. The main objective is to generate predefined trajectories, including a straight line, a circle, and an infinity sign, and execute them using both Simscape simulation and real hardware. Inverse position kinematics is used to convert desired end-effector positions into active joint angles, while forward position kinematics is used for verification. Velocity kinematics and static force analysis are derived using the Jacobian and virtual work principles. The hardware system uses MATLAB for trajectory generation and inverse kinematics, while Arduino executes the motion using two NEMA 17 stepper motors driven by TMC2209 drivers. Experimental results show that the robot can execute drawing trajectories using a MATLAB-to-Arduino serial communication interface.**

***Index Terms*—Pantograph robot, 5R parallel mechanism, inverse kinematics, Simscape, Arduino, stepper motor, trajectory generation.**

I. Introduction

Parallel manipulators are widely used in applications requiring stiffness, compactness, and accurate planar motion. Unlike serial manipulators, parallel mechanisms distribute motion through multiple kinematic chains, which can improve rigidity and reduce accumulated positioning error. This project focuses on a planar pantograph parallel robot designed as a drawing mechanism.

The robot consists of two actuated revolute joints located at the base and two passive distal links connected to a common end-effector point. The objective is to control the end-effector position in the planar workspace and make it follow predefined task-space trajectories. The project includes mechanical design, inverse and forward position kinematics, velocity kinematics, static force analysis, Simscape simulation, and real hardware implementation.

The main contribution of this work is the complete integration between theoretical kinematic modeling, MATLAB trajectory generation, Simscape visualization, and Arduino-based real-time stepper motor execution.

II. Mechanical Design

The pantograph robot is modeled as a planar 5R parallel mechanism consisting of four moving links and a fixed base link connected through revolute joints. The two base joints are actuated, while the remaining joints are passive. The coordinate frame is placed at the midpoint between the two active joints.

The base joint locations are defined as:

$$A_1 = \begin{bmatrix} -\frac{d}{2} \\ 0 \end{bmatrix}, \qquad A_2 = \begin{bmatrix} \frac{d}{2} \\ 0 \end{bmatrix}. \tag{1}$$

The passive joint positions are:

$$B_1 = \begin{bmatrix} -\frac{d}{2} + l_1 \cos\theta_1 \\ l_1 \sin\theta_1 \end{bmatrix}, \tag{2}$$

$$B_2 = \begin{bmatrix} \frac{d}{2} + l_2 \cos\theta_2 \\ l_2 \sin\theta_2 \end{bmatrix}. \tag{3}$$

The end-effector position is:

$$P = \begin{bmatrix} x \\ y \end{bmatrix}. \tag{4}$$

The robot dimensions used in the MATLAB and hardware implementation are listed in Table I.

TABLE I
ROBOT GEOMETRIC PARAMETERS

Parameter	Description	Value
l_1	Left proximal link	12.5 cm
l_2	Right proximal link	12.5 cm
l_3	Left distal link	16.6 cm
l_4	Right distal link	16.6 cm
d	Distance between active joints	5.8 cm

III. Methodology

A. Inverse Position Kinematics

Inverse position kinematics is used to calculate the active joint angles required to reach a desired end-effector point. The input is the desired point $P = [x, y]^T$, and the output is the active joint vector:

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}. \tag{5}$$

The distances from the active joints to the end-effector are:

$$a_1 = \sqrt{\left(x + \frac{d}{2}\right)^2 + y^2}, \tag{6}$$

$$a_2 = \sqrt{\left(x - \frac{d}{2}\right)^2 + y^2}. \tag{7}$$

The direction angles are:

$$\alpha_1 = \tan^{-1}\left(\frac{y}{x + \frac{d}{2}}\right), \tag{8}$$

$$\alpha_2 = \tan^{-1}\left(\frac{y}{x - \frac{d}{2}}\right). \tag{9}$$

Using the law of cosines:

$$\beta_1 = \cos^{-1}\left(\frac{l_1^2 + a_1^2 - l_3^2}{2 l_1 a_1}\right), \tag{10}$$

$$\beta_2 = \cos^{-1}\left(\frac{l_2^2 + a_2^2 - l_4^2}{2 l_2 a_2}\right). \tag{11}$$

For the selected front-half working configuration, the active joint angles are:

$$\theta_1 = \alpha_1 + \beta_1, \tag{12}$$

$$\theta_2 = \alpha_2 - \beta_2. \tag{13}$$

This IPK algorithm is applied to every point in the desired task-space trajectory before sending commands to the simulation or real hardware.

B. Forward Position Kinematics

Forward position kinematics is used to verify the inverse kinematics. Given θ_1 and θ_2, the positions B_1 and B_2 are first calculated. The end-effector position is then obtained from the intersection of two circles:

$$|P - B_1| = l_3, \qquad |P - B_2| = l_4. \tag{14}$$

The front/top intersection is selected because it corresponds to the physical assembly mode of the robot.

C. Velocity Kinematics

The velocity kinematics are derived from the closed-loop constraints:

$$P - B_1 = l_3 \hat{s}_3, \qquad P - B_2 = l_4 \hat{s}_4, \tag{15}$$

where $\hat{s}_3$ and $\hat{s}_4$ are unit vectors along the passive distal links. Differentiating the loop constraints and eliminating the passive angular velocities gives:

$$\hat{s}_3^T(\dot{P} - \dot{B}_1) = 0, \qquad \hat{s}_4^T(\dot{P} - \dot{B}_2) = 0. \tag{16}$$

The final velocity relation is:

$$J_x \dot{\chi} = J_q \dot{q}, \tag{17}$$

where:

$$J_x = \begin{bmatrix} s_{3x} & s_{3y} \\ s_{4x} & s_{4y} \end{bmatrix}, \tag{18}$$

$$J_q = \begin{bmatrix} l_1(-s_{3x}\sin\theta_1 + s_{3y}\cos\theta_1) & 0 \\ 0 & l_2(-s_{4x}\sin\theta_2 + s_{4y}\cos\theta_2) \end{bmatrix}. \tag{19}$$

The general Jacobian is:

$$\dot{q} = J\dot{\chi}, \qquad J = J_q^{-1} J_x. \tag{20}$$

D. Static Force Analysis

Static force analysis was performed using the virtual work principle. At static equilibrium:

$$\delta W = 0. \tag{21}$$

The total virtual work is:

$$0 = \tau^T \delta q - F^T \delta \chi. \tag{22}$$

Using the Jacobian relation:

$$\delta q = J \delta \chi, \tag{23}$$

the static force relation becomes:

$$F = J^T \tau. \tag{24}$$

Thus, the actuator torques required to produce a desired end-effector force are:

$$\tau = J^{-T} F. \tag{25}$$

IV. Trajectory Generation and Implementation

Three task-space trajectories were implemented: a straight line, a circle, and an infinity sign. Each trajectory was generated as a sequence of end-effector points. The IPK function was then used to convert the task-space points into active joint angle commands.

A linear trajectory was generated using:

$$P(t) = P_{start} + \alpha(P_{end} - P_{start}), \qquad 0 \le \alpha \le 1. \tag{26}$$

A circular trajectory was generated using:

$$x = c_x + r\cos\theta, \qquad y = c_y + r\sin\theta. \tag{27}$$

The infinity trajectory used for Simscape-style motion was:

$$x = c_x + \frac{s\cos t}{1 + \sin^2 t}, \tag{28}$$

$$y = c_y + \frac{s\sin t\cos t}{1 + \sin^2 t}. \tag{29}$$

A. Simscape Simulation

The mechanical model was simulated using Simscape Multibody. MATLAB generated the desired trajectory, converted it to joint angles using IPK, and produced time-series inputs for the active revolute joints. Angle offsets were added in Simscape because the revolute joint reference frames did not match the theoretical MATLAB angle convention. These offsets were used only in simulation and were not sent to the real hardware.

B. Hardware Implementation

The real robot used two NEMA 17 stepper motors driven by TMC2209 drivers. MATLAB generated the trajectory and sent the complete list of joint angle commands to Arduino through serial communication. Arduino converted the received angles into step positions and generated synchronized STEP/DIR pulses for both motors.

The serial protocol used was:

$$\texttt{HOME} \rightarrow \texttt{BEGIN} \rightarrow \texttt{P} \rightarrow \texttt{END} \rightarrow \texttt{RUN}. \tag{30}$$

The `HOME` command synchronized Arduino's internal step counters with the known physical home position. The trajectory points were then stored on Arduino before execution. This batch execution method reduced stop-start motion compared to sending one command at a time.

The angle-to-step conversion was:

$$\text{steps per degree} = \frac{200 \times 16 \times G}{360}. \tag{31}$$

With $G = 1$, the conversion becomes:

$$\text{steps per degree} = 8.8889. \tag{32}$$

V. Results

The IPK and FPK models were verified numerically in MATLAB. A test point was passed into the IPK function to calculate the active joint angles, then these angles were passed into the FPK function to recover the original end-effector point. The resulting position error was negligible, confirming the consistency of the kinematic model.

The Simscape simulation successfully visualized the pantograph motion following the desired infinity trajectory. The active revolute joints were driven by time-series inputs generated from the MATLAB IPK model.

The hardware system successfully executed drawing trajectories using the MATLAB-Arduino interface. The line and infinity trajectories were executed on the physical robot. The hardware implementation showed that sending the full trajectory to Arduino before execution improved motion continuity because the Arduino handled real-time step pulse generation locally.

TABLE II
Summary of project results

Item	Result
IPK calculation	Successful
FPK verification	Negligible position error
Simscape simulation	Infinity trajectory achieved
Hardware execution	Line and infinity trajectories achieved
Control architecture	MATLAB trajectory generation + Arduino real-time stepping

VI. Discussion

The project demonstrated the complete workflow of a pantograph parallel robot, starting from kinematic analysis and ending with physical hardware execution. The main challenge in the hardware implementation was the open-loop nature of stepper motor control. Since no encoders were used, the system relies on accurate manual homing and assumes that the motors do not skip steps.

Another challenge was the difference between Simscape and real hardware angle conventions. Simscape required angular offsets due to joint frame orientation, while the real robot used the direct IPK motor angles. Therefore, separate signal conditioning was required for simulation and hardware execution.

The current implementation can be improved by adding encoders, automatic homing sensors, and closed-loop feedback. These additions would reduce position errors caused by manual calibration, missed steps, and mechanical backlash.

VII. Conclusion

A planar pantograph parallel robot was designed, modeled, simulated, and implemented as a drawing system. The inverse position kinematics algorithm successfully converted desired end-effector positions into active motor angles. Forward position kinematics verified the consistency of the position model, while velocity kinematics and static force analysis were derived using the Jacobian.

The Simscape model confirmed the feasibility of the desired motion, and the hardware implementation demonstrated trajectory execution using MATLAB and Arduino. The final system was able to draw predefined trajectories, including a line and an infinity sign. The project successfully integrated mechanical design, kinematic analysis, simulation, and real-time hardware control.

References

[1] GIU Parallel Kinematics and Kinetics, *Tutorial 8: Velocity Kinematics*.

[2] GIU Parallel Kinematics and Kinetics, *Tutorial 9: Static Forces and Virtual Work*.

[3] Northwestern University Mechatronics Wiki, "Design and Control of a Pantograph Robot," 2010. [Online]. Available: https://hades.mech.northwestern.edu/index.php/Design_and_Control_of_a_Pantograph_Robot.

[4] M. D. Ratolikar and P. K. R., "Optimized design of 5R planar parallel mechanism aimed at gait-cycle of quadruped robots," *Journal of Vibroengineering*, 2021, doi: https://doi.org/10.21595/jve.2021.22131.